

Sample Project Documentation

AI-ML & GEN-AI Track | Project Phase Documentation

Date	30 September 2026	Team ID	SWTID-2026-8540
Project Name	AI BlogNest API	Track	AI / ML / Generative AI
Team Size	5	Maximum Marks	Project Documentation
Team Leader	Pravin kumar .P	Members	Abdul Rahaman .A; Dhamotharan .G; Maharaja .C; SUBHASH S

1. Project Summary

AI BlogNest API is a secure, modular REST API platform designed to simplify blog creation, content management and AI-assisted writing. It uses Node.js and Express.js, MongoDB with Mongoose, Gemini AI integration, JWT authentication and role-based access control.

2. Key Capabilities

- AI-powered blog content generation
- AI-assisted summarization and FAQ support
- Blog CRUD with lifecycle states
- Categories, tags, comments and media metadata
- JWT authentication and Admin / Editor / Author / Reader roles
- Validation, sanitization, rate limiting and structured logging

3. Scenario-Based Case Study

A content author needs to create a draft from a topic, organize it with categories and tags, submit it for review and later publish it. Editors and administrators require protected access for review and moderation. Readers need public access to published content. AI services reduce repetitive drafting and summarization work while human review remains part of the workflow.

4. Software Requirements

Requirement	Details
Operating System	Windows 10/11, macOS or Linux
Runtime	Node.js v16+
Package Manager	npm v8+
Framework	Express.js
Database	MongoDB
ODM	Mongoose
API Testing	Postman / Thunder Client
Code Editor	Visual Studio Code or equivalent

5. Architecture



6. Data Model

Entity	Representative Fields
User	_id, name, email, password, role
Post	_id, userID, name/title, photo, message, likes, timestamps
Comment	_id, post reference, user reference, content/moderation state
Taxonomy / Media Metadata	category, tags, media/path metadata as required by implementation

7. API Flow

Typical request flow: Client -> Express route -> JWT / RBAC / validation middleware -> controller -> service logic -> MongoDB or Gemini AI integration -> standardized JSON response. AI generation and summarization are represented as dedicated service endpoints in the reference architecture.

8. Security Controls

Control	Implementation Direction
Authentication	JWT for stateless authentication
Password Storage	bcrypt.js hashing
Authorization	Role-based route guarding
Input Protection	Validation and sanitization
Abuse Protection	Rate limiting
Operational Safety	Centralized errors and structured logs

9. Testing Approach

Verify authentication, protected routes, CRUD operations, AI generation, summarization, role boundaries, invalid input handling, comments and rate limiting using API testing tools. Capture request/response evidence for final submission.

10. Maintenance and Scaling

Keep domain logic modular, optimize database access, isolate AI service calls and extend monitoring/analytics as usage grows. Future features can include richer analytics, scheduled publishing enhancements, collaborative editing and additional AI assistance.

11. Demo Checklist

Step	Demo Action	Expected Output
1	Login as Author	JWT returned
2	Create or generate a draft	Blog record created / AI response returned
3	Add category / tag	Organized content
4	Review role behavior	Unauthorized action blocked
5	Summarize article	Concise AI-generated summary
6	Show Reader flow	Published content available
7	Show logs / error handling	Controlled operational response

12. Team and Responsibilities

Member	Primary Area
Pravin kumar .P	Team coordination, architecture and backend integration
Abdul Rahaman .A	Requirements, API workflow and content modules
Dhamotharan .G	AI-assisted features and documentation
Maharaja .C	Security, validation and testing
SUBHASH S	Database/content support, testing and demonstration

13. Conclusion

AI BlogNest API provides a structured foundation for an AI-assisted blogging backend. Its layered architecture separates routing, authorization, business logic, database access and AI integration, making the solution easier to maintain and extend.

14. Reference Architecture Notes

The project documentation is aligned with the supplied reference document: Node.js + Express.js, MongoDB + Mongoose, Gemini AI, JWT, bcrypt.js, role-based permissions, content CRUD, comments, taxonomy, media metadata, search, validation, sanitization, rate limiting and logging.